

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115**

 Bug **15**

 Code Smell **100**

Tags

▼

Impact

▼

Clean code attribute

▼

Search by name...

🔍

Function declarations should be preferred over variables

 Code Smell

Extension identifiers should comply with a naming convention

 Code Smell

Local identifiers should not start with underscore

 Code Smell

If-null operator should be preferred

 Code Smell

Null-aware operators should be preferred

 Code Smell

Generic function type aliases should be preferred

 Code Smell

Type parameters should not shadow other type parameters

 Code Smell

"static final" declarations should be "const" instead

 Code Smell

Interpolation should be used instead of String concatenation

 Code Smell

Collection literals should be preferred

 Code Smell

Conditional assignment should be preferred

 Code Smell

"this" should only be used when required

Function declarations should be preferred over variables

Analyze your code

Consistency - Conventional

Maintainability 

 Code Smell  Minor  convention

Function declarations should be used, instead of variables.

Why is this an issue?

How can I fix it?

More Info

Dart has **built-in support for functions**, meaning that they can be assigned to variables like any other type.

However, this leads to two ways of defining functions:

- function declarations, both in top-level and nested scopes
- and function expressions assigned to variables, which can be library-level variables or local ones

The second way allows to redefine the function behavior after first assignment, by simply reassigning the variable to a new function expression. When this is not the case, the function expression should be replaced with a function declaration, to make the code more readable and consistent.

Exceptions

Top-level variables are allowed, since they can change their value over time.

```
typedef StringRetriever = String Function(Map<String, String> data);
StringRetriever currentRetriever = (data) => '';
```

However, top-level `final` variables are not allowed, since their value cannot change once initialized. Therefore, there is no reason not to define them as function expressions.

```
final StringRetriever currentRetriever = (data) => ''; // Noncompliant
```

Nested function variables, on the other hand, are always considered non-compliant, whether they are `final` or not, but only if their values is not mutated.

```
void main() {
  StringRetriever mutableRetriever = (data) => ''; // Noncompliant
  final StringRetriever immutableRetriever = (data) => ''; // Noncompliant

  var nestedFunctionVariable = () { }; // Compliant
  nestedFunctionVariable = () { print('Hi'); };
}
```

The same applies to class fields (instance and static):

```
class AClass {
  final StringRetriever immutableFunctionField = (data) => ''; // Noncompliant
  StringRetriever mutableFunctionField = (data) => ''; // Compliant

  static final StringRetriever immutableStaticFunctionField = (data) => ''; // Noncompliant
  static StringRetriever mutableStaticFunctionField = (data) => ''; // Compliant
}
```

Available In:

sonarcloud 

